

tebei

Text · Image · Retrieval Engine

Project Documentation

Information Retrieval — Final Project 2025/2026

Faculty of Computer Science · Universitas Indonesia

Name	NPM	Role
Muhammad Hafiz	2206082732	CLIP service, FAISS index, Rocchio PRF, architecture
Ratu Nadya Anjania	2206029752	Cross-encoder reranker, upload pipeline
Wahyu Hidayat	2206081894	Search suggestions, Patricia trie autocomplete

<https://tebei.hafizmuh.site>

Table of Contents

1. Overview

2. System Architecture

3. Retrieval Pipelines

3.1 Text-to-Image Pipeline

3.2 Image-to-Image Pipeline

4. Technology Stack

5. Dataset & Index

6. Module Reference

6.1 app/main.py

6.2 app/core/config.py

6.3 app/models/schemas.py

6.4 app/services/clip_service.py

6.5 app/services/data_store.py

6.6 app/services/query_expansion.py

6.7 app/services/text_retrieval.py

6.8 app/services/image_retrieval.py

6.9 app/services/reranker.py

6.10 app/api/routes/search.py

6.11 app/api/routes/upload.py

7. API Reference

8. Getting Started

9. Design Decisions

10. Known Limitations

1. Overview

tebei is a multimodal information retrieval system that enables users to search a corpus of ~31,000 photographs using either natural language descriptions or a query image. It was built as the final project for the Information Retrieval course (2025–2026) at the Faculty of Computer Science, Universitas Indonesia.

The system combines several retrieval strategies into a single hybrid pipeline: semantic vector search via CLIP embeddings, lexical matching via BM25, LLM-based query expansion using Gemini 2.5 Flash, Rocchio pseudo-relevance feedback, and cross-encoder reranking. All components are served through a FastAPI application deployed in a Docker container.

Key Capabilities

- **Text-to-image search** — Natural language query → ranked image results
- **Image-to-image search** — Upload a photo → find visually similar images
- **LLM query expansion** — Gemini generates visual descriptions to enrich abstract queries
- **Pseudo-relevance feedback** — Rocchio algorithm refines the query vector using top results
- **Hybrid retrieval** — CLIP semantic search + BM25 lexical search, fused with RRF
- **Cross-encoder reranking** — ms-marco-MiniLM rescores (query, caption) pairs
- **Multilingual queries** — Automatic translation to English before CLIP encoding
- **Autocomplete** — O(k) prefix lookup via a Patricia tree over caption vocabulary
- **Image upload** — New images are encoded and inserted into the live FAISS index
- **Rate limiting** — Per-IP throttling via slowapi

2. System Architecture

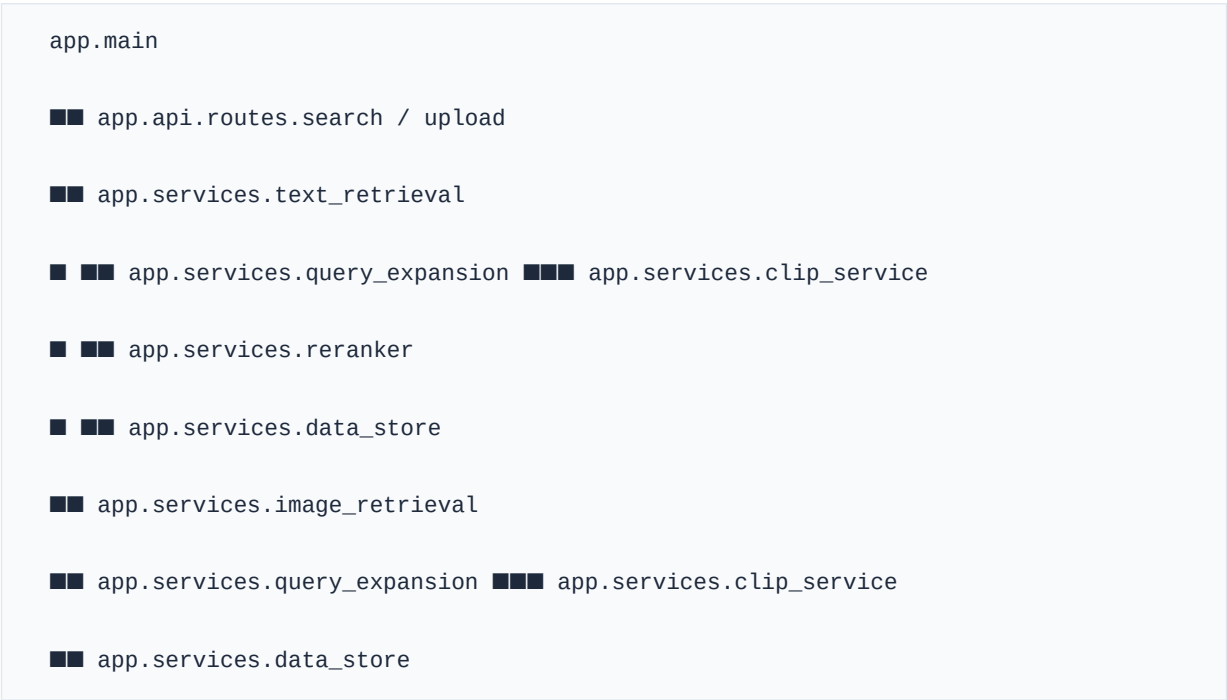
The application follows a service-oriented layered architecture. The browser communicates with a FastAPI application over HTTP. The application layer routes requests to a services layer, which contains all retrieval logic. A shared **CLIPService** singleton holds the model and FAISS index in memory, accessed by all services without re-loading.

Layer	Components	Responsibility
Frontend	Vanilla JS + Fetch API, Jinja2	Search UI, image upload, autocomplete, modals
API	FastAPI routes + slowapi	Request routing, validation, rate limiting
Services	TextRetriever, ImageRetriever, DataStore, Reranker	Retrieval pipelines, BM25, trie
Shared core	CLIPService	CLIP model, FAISS IndexFlatIP, negative matrix
Storage	data/index/, data/raw/	FAISS index, image files, captions.txt

Startup Sequence

On startup, FastAPI's lifespan context manager calls **CLIPService.is_ready()**, which loads the CLIP model into RAM and reads the FAISS index and image-ID list from disk. Concurrently, DataStore initialises on import: it parses captions.txt, builds the BM25 index, computes search suggestions, and populates the Patricia trie. All structures are ready before the first request is accepted.

Dependency Graph



3. Retrieval Pipelines

3.1 Text-to-Image Pipeline

A text query passes through five sequential stages before results are returned. Each stage is designed to degrade gracefully: if translation or LLM expansion fails, the pipeline continues with the unenriched query.

Stage	Component	Detail
1 — Translate	GoogleTranslator (deep-translator)	Auto-detect source language; translate to English. Fails silently — returns original query on error.
2 — LLM Expansion	Gemini 2.5 Flash + CLIP encode	Gemini generates 3 visual descriptions. All 4 texts (query + descriptions) are encoded with CLIP and averaged into a single enriched query vector.
3 — Rocchio PRF	FAISS search + dot-product	positive = top-10 most similar vectors (FAISS search) negative = top-10 most dissimilar vectors (brute-force) refined = $\alpha \cdot q + \beta \cdot \text{mean}(\text{pos}) - \gamma \cdot \text{mean}(\text{neg})$, L2-normalised $\alpha=1.0$ $\beta=0.75$ $\gamma=0.15$
4 — Hybrid Retrieval	FAISS IndexFlatIP + BM25 Okapi	Semantic: cosine similarity via inner product on L2-norm vectors. Lexical: BM25 Okapi score over tokenised caption text. RRF fusion: $\text{score}(d) = \sum 1/(60 + \text{rank}_i(d))$
5 — Reranking	ms-marco-MiniLM CrossEncoder	Scores every (query, caption) pair in a single batch. Sigmoid-normalised to (0, 1). Falls back to RRF order if top score < 0.1 (weak matches across the board).

3.2 Image-to-Image Pipeline

Image search is a simplified three-stage pipeline that skips translation, LLM expansion, and cross-encoder reranking, since there is no text query to enrich or caption to compare against.

Stage	Component	Detail
1 — CLIP Encode	CLIPService.encode_image	Load image bytes → PIL RGB conversion → CLIP forward pass. Outputs a single L2-normalised 512-dim float32 vector.
2 — Rocchio PRF	FAISS + brute-force dot product	Same formula as text pipeline ($\alpha=1.0$, $\beta=0.75$, $\gamma=0.15$). Falls back to unrefined vector when image_matrix.npy is absent.
3 — FAISS Search	IndexFlatIP	Exact inner-product search on the refined vector. Returns top-k (image_id, cosine_score) pairs.

4. Technology Stack

Component	Library / Version	Reason for choice
Backend framework	FastAPI 0.115 + Uvicorn 0.32	Async, type-safe, auto-generates OpenAPI docs
Embedding model	CLIP ViT-B/32 (sentence-transformers)	One model encodes text + images into same 512-dim space
Vector index	FAISS IndexFlatIP (faiss-cpu 1.9+)	Exact cosine search; 100% recall at 31k-vector scale
Lexical search	BM25 Okapi (rank-bm25 0.2+)	Handles rare keywords and exact term matches
Reranker	ms-marco-MiniLM-L-6-v2	Cross-attention over (query, caption) pairs
LLM expansion	Gemini 2.5 Flash (REST API)	Generates visual descriptions for abstract queries
Translation	deep-translator 1.11 (Google Translate)	Enables multilingual queries without multilingual CLIP
Autocomplete	Custom Patricia tree	O(k) prefix lookup, memory-efficient
Rate limiting	slowapi 0.1.9	Per-IP throttling per endpoint
Stop words	NLTK 3.8+ English stop words	Filters noise from BM25 and trie vocabulary
Config	pydantic-settings 2.6	Type-safe settings from environment variables
Container	Docker + docker-compose	Reproducible build and deployment

5. Dataset & Index

Flickr30k Dataset

The system indexes the **Flickr30k** dataset, which contains 31,783 photographs sourced from the Flickr photo sharing platform. Each image is annotated with five independently written English captions from crowd workers, yielding 158,915 caption sentences in total. The dataset covers a wide variety of subjects: people, animals, outdoor scenes, sports, and everyday activities.

Statistic	Value
Total images	31,783
Captions	158,915 (5 per image)
Embedding dim	512 (CLIP ViT-B/32)
Index type	FAISS IndexFlatIP
Index file size	~63 MB (image_index.faiss)
Vector dtype	float32, L2-normalised

Index Build Process

Run **scripts/build_index.py** once after placing the dataset. The script performs the following steps:

- Parse captions.txt — reads the CSV, one row per caption.
- Encode images — each image is opened, converted to RGB, and passed through CLIP. Embeddings are L2-normalised before insertion.
- Build FAISS index — vectors are added to IndexFlatIP and serialised to image_index.faiss.
- Save raw matrix — all vectors are stacked into image_matrix.npy for Rocchio negative feedback.
- Serialise IDs — the ordered image filename list is written to image_ids.json.
- Build BM25 and Patricia trie — DataStore initialises automatically on import.

Index Files

```
data/index/

image_index.faiss # FAISS IndexFlatIP serialised to disk

image_ids.json # ordered list of image filenames

image_matrix.npy # raw float32 matrix for Rocchio negative feedback
```

6. Module Reference

6.1 app/main.py

Purpose

FastAPI application entry point. Sets OpenMP environment variables before any PyTorch import to prevent a crash on macOS 26+ ARM64 (known libomp bug). Registers the lifespan context manager, mounts static and image directories, attaches the rate limiter, and includes the search and upload routers.

Key items

`lifespan()` — loads CLIP model at startup. `home()` — serves the single-page UI. `health()` — liveness probe for Docker health checks.

6.2 app/core/config.py

Purpose

Typed application settings backed by pydantic-settings. Values are read from environment variables or a `.env` file, with sensible defaults for local development.

Settings fields

`APP_NAME`, `DATA_DIR`, `IMAGES_DIR`, `CAPTIONS_FILE`, `INDEX_DIR`, `MAX_UPLOAD_SIZE` (5 MB), `GEMINI_API_KEY`.

6.3 app/models/schemas.py

Purpose

Pydantic request/response schemas used by the search endpoints.

Models

`SearchRequest` — query string and `top_k`. `SearchResult` — `image_id`, `image_url`, `score`, `caption`. `SearchResponse` — query, results list, total count.

6.4 app/services/clip_service.py

Purpose

Singleton wrapper around the CLIP model and FAISS index. A single clip-ViT-B-32 SentenceTransformer model encodes both text and images into the same 512-dimensional L2-normalised space, saving ~600 MB of RAM compared to running separate text and image models.

Key methods

`is_ready()` — lazy-loads the model and index, returns True when the index is non-empty. `encode_text(query)` / `encode_texts(queries)` — CLIP text encoding. `encode_image(bytes)` — CLIP image encoding. `search_by_embedding(emb, top_k)` — inner-product nearest-neighbour search. `search_negative(emb, top_k)` — brute-force dot product to find most-dissimilar vectors. `get_vectors(indices)` — reconstruct stored vectors for Rocchio centroid computation. `add_image(bytes, id)` — encode, insert, and persist to disk.

6.5 app/services/data_store.py

Purpose

In-memory store for captions, BM25 index, search suggestions, and the Patricia trie. Initialised at import time by reading captions.txt.

Key components

`PatriciaTrie` — compressed radix trie providing $O(k)$ prefix autocomplete. `BM25Okapi` — lexical index over concatenated captions per image. `suggestions` — top bigrams and unigrams by frequency, served to the UI. `add_image()` — appends a new caption row to captions.txt and updates the in-memory map.

6.6 app/services/query_expansion.py

Purpose

Implements two query expansion strategies: LLM expansion (Gemini) and Rocchio pseudo-relevance feedback. Both strategies are applied together in the text pipeline; only Rocchio is used in the image pipeline.

Rocchio parameters

$\alpha=1.0$ (original query weight), $\beta=0.75$ (positive centroid weight), $\gamma=0.15$ (negative centroid weight). $K_FEEDBACK=10$ positive examples, $K_NEGATIVE=10$ negative examples.

LRU cache

`_call_gemini()` is decorated with `lru_cache(maxsize=256)`. Repeated or popular queries never make a second API call.

Key functions

`expand_with_llm(query, top_k)` — full text expansion pipeline. `expand_image_query(bytes, top_k)` — image PRF pipeline.

6.7 app/services/text_retrieval.py

Purpose

Orchestrates the end-to-end text-to-image retrieval pipeline.

Pipeline stages

1. `_translate_to_english()` — auto-detect + translate. 2. `expand_with_llm()` — LLM + Rocchio PRF → FAISS hits. 3. `data_store.search_bm25()` — lexical hits. 4. `_fuse_results()` — RRF fusion ($k=60$). 5. `reranker.rerank()` — cross-encoder reranking. Fallback: if top reranker score < 0.1 , returns RRF-fused order.

6.8 app/services/image_retrieval.py

Purpose

Thin orchestrator for the image-to-image retrieval pipeline.

Pipeline

Delegates to `expand_image_query()` (CLIP encode → Rocchio PRF → FAISS search) and formats the results into the standard response dict.

6.9 app/services/reranker.py

Purpose

Cross-encoder reranker using ms-marco-MiniLM-L-6-v2. The model is loaded lazily on first use. Raw logits are sigmoid-normalised to (0, 1).

rerank(query, candidates, captions, top_k)

Builds (query, caption) pairs for every candidate, predicts scores in a single batch, and returns the top-k results sorted by descending score.

6.10 app/api/routes/search.py

Endpoints

POST /api/search/text — text-to-image search, 15 req/min/IP. POST /api/search/image — image-to-image search, 10 req/min/IP. GET /api/search/suggestions — popular search terms. GET /api/search/autocomplete?q= — prefix autocomplete via trie.

6.11 app/api/routes/upload.py

Endpoint

POST /api/upload/image — 30 req/min/IP.

Behaviour

Validates file size ($\leq$ MAX_UPLOAD_SIZE) and extension (.jpg, .jpeg, .jpe, .jfif, .png, .webp). Assigns a UUID-based filename, saves the file, encodes it with CLIP, inserts it into the live FAISS index, and appends the caption to captions.txt.

7. API Reference

The base URL for all endpoints is **http://localhost:8000** (local) or **https://tebei.hafizmuh.site** (production). Interactive documentation is available at **/docs** (Swagger UI) and **/redoc**.

Method	Path	Description	Rate limit
GET	/	Single-page search UI	—
POST	/api/search/text	Text-to-image search	15/min/IP
POST	/api/search/image	Image-to-image search (multipart)	10/min/IP
GET	/api/search/suggestions	Popular search term list	30/min/IP
GET	/api/search/autocomplete?q=	Prefix autocomplete results	30/min/IP
POST	/api/upload/image	Upload image + caption to index	30/min/IP
GET	/api/health	Liveness probe — returns {status:ok}	—
GET	/docs	Swagger UI (auto-generated)	—

POST /api/search/text — Request Body (JSON)

```
{  
  
  "query": "a dog running on the beach",  
  
  "top_k": 12  
}
```

Search Response Schema

```
{  
  
  "query": "a dog running on the beach",  
  
  "total": 12,  
  
  "results": [  
  
    {  
  
      "image_id": "3637013_4d4b3c7c2a.jpg",  
  
      "image_url": "/images/3637013_4d4b3c7c2a.jpg",  
  
    }  
  ]  
}
```

```
"score": 0.4821,  
  
"caption": "A brown dog runs across a sandy beach."  
  
},  
  
...  
  
]  
  
}
```

8. Getting Started

Prerequisites

- Python 3.10+ (local) or Docker (production)
- Flickr30k dataset — download from Kaggle (srinivasac/flickr30k-dataset)
- Gemini API key from Google AI Studio — optional, enables LLM query expansion

Local Development

```
# 1. Clone and create virtual environment

git clone && cd tebei

python3 -m venv env && source env/bin/activate


# 2. Install dependencies

pip install -r requirements.txt


# 3. Place dataset files

mkdir -p data/raw/Images

# copy all .jpg images to data/raw/Images/

# copy captions.txt to data/raw/captions.txt


# 4. Configure environment

cp .env.example .env

# edit .env: set GEMINI_API_KEY=your-key-here


# 5. Build the FAISS index (run once, ~20 min on CPU)

python scripts/build_index.py


# 6. Start the development server

make run # → http://localhost:8000
```

Docker (Production)

```
docker compose build app

docker compose up -d

# → http://localhost:8001
```

Makefile Commands

Command	Description
make install	Create venv and install all dependencies
make run	Start dev server with hot-reload on port 8000
make test	Run pytest test suite
make lint	Run ruff linter on app/ and tests/
make build-index	Trigger index rebuild via API endpoint

9. Design Decisions

Single CLIP model for text and images

CLIP ViT-B/32 maps both modalities into the same 512-dimensional space, so a single model serves all encoding needs. Running separate text and image models would consume ~600 MB of additional RAM with no retrieval quality benefit.

FAISS IndexFlatIP over HNSW

Exact inner-product search on ~31k L2-normalised vectors is fast enough at this scale (sub-millisecond per query) and provides 100% recall. HNSW would give ~99% approximate recall while adding implementation complexity. IndexFlatIP also supports vector reconstruction via `index.reconstruct()`, which is required for Rocchio centroid computation.

Negative feedback in Rocchio

Standard Rocchio uses only positive pseudo-relevance feedback. Adding a negative centroid ($\gamma \cdot \text{mean}(\text{bottom-K vectors})$) explicitly pushes the refined query vector away from the most dissimilar images, improving precision on ambiguous or broad queries.

LRU cache on Gemini calls

LLM expansion is the slowest step (~1–2 s). Because the same query string always produces the same prompt, the Gemini API response is effectively deterministic. `lru_cache(256)` eliminates redundant API calls for repeated or popular queries without any database.

Cross-encoder fallback threshold

If the top cross-encoder score after sigmoid normalisation is below 0.1, all (query, caption) pairs are weak matches. Returning the reranked list in this case would arbitrarily reorder weak results. The system falls back to the RRF-fused ranking, which is more stable under low-confidence conditions.

Patricia tree for autocomplete

A compressed radix trie stores only branching points, reducing memory compared to a full character-level trie. Prefix lookup is $O(k)$ where k is the query length, independent of vocabulary size — important when indexing hundreds of thousands of unique caption tokens.

10. Known Limitations

Limitation	Description
Caption dependency	The cross-encoder scores (query, caption) text pairs. Images with short or missing captions may rank lower regardless of visual relevance. CLIP semantic search partially compensates, but caption quality is a hard constraint on reranking accuracy.
LLM availability	Query expansion depends on the Gemini API. When the API is unreachable or GEMINI_API_KEY is empty, the system silently falls back to PRF-only CLIP search. Results remain reasonable but may be less precise for abstract or ambiguous queries.
English-only CLIP	The ViT-B/32 CLIP model was trained primarily on English text. Translation is applied before encoding, but translation errors propagate directly to retrieval quality, particularly for language-specific or culturally specific queries.
In-process BM25	rank-bm25 runs entirely in memory with whitespace tokenisation. A production-scale system would benefit from an inverted index on disk, stemming, and phrase query support.
No incremental index rebuild	The build_index.py script processes the entire dataset. After uploading many new images via the API, image_matrix.npy becomes stale (it only covers the original build). Rocchio negative feedback will degrade proportionally for newly added images until the index is rebuilt.